SOLID – Interface Segregation Principle (ISP)

Définition

Le principe de ségrégation des interfaces (Interface Segregation Principle — ISP) énonce que les clients ne doivent pas être forcés de dépendre d'interfaces qu'ils n'utilisent pas. Autrement dit, une interface doit être spécifique à un rôle donné et ne contenir que les méthodes pertinentes pour les classes qui l'implémentent.

Motivation

Dans un système orienté objet, il est fréquent de vouloir créer des interfaces génériques regroupant de nombreuses opérations. Cependant, ces 'grosses' interfaces imposent souvent à certaines classes d'implémenter des méthodes inutiles, créant un couplage inutile et rendant le code fragile.

Problème des 'fat interfaces'

Une interface trop large :
• Force certaines classes à implémenter des méthodes vides ou non pertinentes.
• Rend le code difficile à maintenir et à faire évoluer.
• Crée un couplage entre des modules qui ne devraient pas dépendre les uns des autres.
• Va à l'encontre du principe de responsabilité unique (SRP).

Exemple simplifié

```
interface Worker {
void work();
void eat();
}

class Robot implements Worker {
public void work() { ... }
public void eat() { /* inutile pour un robot */ }
}
```

➡ Ici, le robot est forcé d'implémenter une méthode 'eat' qui n'a pas de sens pour lui. La solution consiste à scinder l'interface :

```
interface Workable { void work(); }
```

interface Eatable { void eat(); }
Ainsi, chaque classe implémente uniquement l’interface correspondant à son rôle.

Avantages du respect de l’ISP

■ Réduction du couplage entre composants
■ Interfaces plus cohérentes et expressives
■ Meilleure lisibilité et maintenabilité du code
■ Facilite les tests unitaires et l’extension du système

Bonnes pratiques

• Concevoir des interfaces petites et spécifiques à un rôle.
• Éviter les implémentations vides ou inutiles : elles indiquent souvent une violation de l’ISP.
• Utiliser plusieurs interfaces complémentaires plutôt qu’une seule interface générique.
• Combiner les interfaces par composition ou héritage lorsque c’est logique.

Résumé

L’ISP encourage la création d’interfaces ciblées et cohérentes. Il complète le principe de responsabilité unique (SRP) en s’assurant que chaque interface exprime une seule intention claire.

➡ En respectant l’ISP :
- les classes sont plus simples,
- les dépendances plus explicites,
- et l’évolution du code devient plus sûre.